

1. Características Básicas del Lenguaje

JavaScript es el lenguaje de programación fundamental de la web moderna. Sus características principales incluyen:

- **Interpretado / JIT (Just-in-Time):** A diferencia de los lenguajes compilados tradicionales (como C++), JavaScript no requiere un paso de compilación previo por parte del desarrollador. El navegador lee el código fuente directamente y, mediante motores modernos (como V8 de Chrome), lo compila al vuelo a código máquina justo antes de ejecutarlo para mejorar el rendimiento.
- **Tipado Dinámico:** Las variables en JavaScript no están fuertemente ligadas a un tipo de dato específico. Esto significa que una misma variable puede contener un número en un momento y una cadena de texto más adelante, sin necesidad de declarar explícitamente su tipo (a diferencia de Java o C#).
- **Multiparadigma:** Es un lenguaje flexible que permite a los desarrolladores adoptar diferentes estilos de programación según la necesidad del proyecto. Soporta la **Programación Orientada a Objetos (POO)** (basada en prototipos), la **Programación Funcional** (tratando a las funciones como ciudadanos de primera clase) y la **Programación Estructurada/Imperativa**.

2. Estructuras de Control

Permiten controlar el flujo de ejecución del código dependiendo de condiciones o repeticiones.

- **Condicionales (if, else if, else, switch):** Evalúan una expresión booleana (verdadero o falso) para decidir qué bloque de código ejecutar. El `switch` se utiliza cuando se tienen múltiples bifurcaciones basadas en el valor de una sola expresión.
- **Bucles (for, while, do-while):** Permiten repetir un bloque de código múltiples veces.
 - `for`: Ideal cuando se conoce de antemano el número de iteraciones.
 - `while`: Se ejecuta mientras la condición especificada sea verdadera.
 - `do-while`: Similar al `while`, pero garantiza que el código se ejecute al menos una vez antes de evaluar la condición.

3. Tipos de Datos

En JavaScript los tipos de datos se dividen en dos categorías principales:

- **Tipos Primitivos:** Son valores básicos e inmutables (no se pueden modificar, solo reemplazar). Se almacenan directamente en la pila (*stack*) de memoria.
 - Number: Para valores numéricos (enteros y decimales).
 - String: Para cadenas de texto.
 - Boolean: Valores lógicos (true o false).
 - Undefined: Indica que una variable ha sido declarada pero aún no se le ha asignado un valor.
 - Null: Representa explícitamente la ausencia intencional de cualquier valor.
 - Symbol y BigInt: Para identificadores únicos y números enteros de gran precisión, respectivamente.
- **Tipos de Referencia (Objetos):** Estructuras complejas que pueden almacenar colecciones de valores o entidades más complejas. No se guarda el valor directamente en la variable, sino una dirección de memoria (referencia) que apunta al lugar donde están guardados en el montón (*heap*). Incluye Objects, Arrays y Functions.

4. Funciones

Las funciones son bloques de código diseñados para realizar una tarea específica y pueden ser invocadas en cualquier momento.

- **Funciones Declaradas:** Se definen utilizando la palabra clave `function` seguida de un nombre. Tienen la propiedad de *hoisting*, lo que significa que JavaScript las eleva internamente al inicio del script y pueden ser invocadas incluso antes de la línea donde fueron escritas.
- **Funciones Expresadas:** Se definen asignando una función anónima (o nombrada) dentro de una variable o constante. A diferencia de las declaradas, no sufren de *hoisting*, por lo que no se pueden usar antes de su definición en el flujo del código.
- **Funciones Flecha (Arrow Functions):** Introducidas en ECMAScript 6, ofrecen una sintaxis mucho más corta y limpia (utilizando `=>`). Además de la estética, tienen una diferencia técnica crucial: no crean su propio contexto para la palabra clave `this`, sino que heredan el `this` del entorno léxico en el que fueron creadas.

5. Objetos y Arrays

Son las estructuras de datos compuestas más utilizadas en el lenguaje.

- **Objetos:** Colecciones de propiedades representadas por pares de clave: valor. Las claves son cadenas de texto (o símbolos) y los valores pueden ser cualquier tipo de dato, incluyendo otras funciones (a las que se les llama métodos). Son ideales para modelar entidades del mundo real.

- **Arrays (Arreglos):** Listas ordenadas de elementos. Cada elemento tiene una posición numérica asignada llamada índice, la cual comienza siempre en 0. JavaScript permite que un solo array contenga diferentes tipos de datos mezclados y provee métodos nativos avanzados para manipularlos (como `.map()`, `.filter()`, `.reduce()`).

6. Eventos y Manipulación del DOM

Es la característica que permite hacer interactiva a una página web.

- **DOM (Document Object Model):** Es una interfaz de programación que representa el documento HTML como un árbol de nodos. JavaScript utiliza el DOM para acceder, modificar, crear o eliminar elementos, etiquetas, atributos y estilos CSS en tiempo real.
- **Eventos:** Son acciones o sucesos que ocurren en el sistema (ya sea provocados por el usuario como un click, mover el mouse, presionar una tecla, o por el navegador como terminar de cargar la página). JavaScript "escucha" estos eventos mediante *Event Listeners* y ejecuta funciones específicas en respuesta.

7. Asincronía en JavaScript

JavaScript es monohilo (ejecuta una sola tarea a la vez), pero maneja operaciones pesadas (como peticiones HTTP o temporizadores) de forma asíncrona mediante el *Event Loop* para no bloquear la experiencia del usuario.

- **Callbacks:** Funciones que se pasan como argumentos a otras funciones para ser ejecutadas una vez que finalice una tarea asíncrona. Su abuso histórico llevó al problema conocido como *Callback Hell* (código anidado y difícil de leer).
- **Promesas (Promises):** Objetos que representan el estado final (ya sea éxito con `.then()` o fallo con `.catch()`) de una operación asíncrona. Pueden estar en tres estados: *Pending* (pendiente), *Fulfilled* (cumplida) o *Rejected* (rechazada).
- **Async / Await:** Una capa de azúcar sintáctico construida sobre las promesas. La palabra clave `async` define que una función es asíncrona, y `await` pausa la ejecución de la función de manera limpia hasta que la promesa se resuelva, permitiendo escribir código asíncrono que se lee de forma lineal y síncrona. Los errores se manejan fácilmente con bloques tradicionales `try / catch`.

Cuadro Sinóptico

Kevin Rivera González

JAVASCRIPT

CARACTERISTICAS

Interpretado: El navegador lee y ejecuta el código directamente, sin la necesidad de haberlo hecho antes.

Dinámico: Las variables pueden cambiar de tipo de dato sobre la marcha.

Multiparadigma: Soporta programación orientada a objetos, funcional y estructurada

ESTRUCTURAS DE CONTROL

Condicionales: Toman decisiones en el código (if, else if, else, switch).

Bucles: Repiten bloques de código (for, while, do-while)

TIPOS DE DATOS

Primitivos: Valores inmutables y directos (Number, String, Boolean, Undefined, Null, Symbol, BigInt).

De referencia: Estructuras más complejas que se guardan una referencia en memoria (Objects, Arrays, Functions).

FUNCIONES

Declaradas: Tienen nombre y se pueden usar antes de definirse gracias al hoisting (Ej: `function suma() {}`).

Expresadas: Se guardan dentro de una variable (Ej: `const suma = function() {}`).

Flecha (Arrow functions): Sintaxis corta y moderna; no tienen su propio contexto de `this` (Ej: `const suma = () => {}`).

OBJETOS Y ARRAYS

Objetos: Colecciones de datos organizados por pares de clave: valor (Ej: `{nombre: "Juan", edad: 22}`).

Arrays: Listas ordenadas de elementos indexados que empiezan desde la posición 0 (Ej: `[1, 2, 3]`).

EVENTOS DE MANIPULACION DEL DOM

DOM (Document Object Model): La estructura del HTML que JavaScript puede modificar (cambiar textos, estilos, crear etiquetas).

Eventos: Acciones del usuario o del navegador (hacer click, escribir, cargar la página) que activan funciones.

ASINCRONIA

Callbacks: Funciones que pasan como argumentos a otra función para ejecutarse cuando termine la tarea.

promesas: Objetos que representan el éxito (resolve) o fallo (reject) de una operación asíncrona a futuro.

Async/Await: Sintaxis moderna que hace que el código asíncrono parezca síncrono, facilitando su lectura usando `try/catch`.